	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): 7

Integrante(s): 323078401

323115296

323082392

323139441

323253916

No. de lista o brigada: 04

Semestre: 2027-I

Fecha de entrega: 16/10/2026

Observaciones: _____

_____

CALIFICACIÓN: _____

Índice

1. Introducción **2**

1.1. Planteamiento del problema 2

1.2. Motivación 2

1.3. Objetivos 2

2. Marco Teórico **2**

3. Desarrollo **3**

3.1. Actividad 1: Análisis de las respuestas de las LLMs 3

3.2. Actividad 2: Programas de la jerarquía de cuadriláteros 5

4. Resultados **7**

4.1. Resultados de la Práctica 7 7

5. Conclusiones **9**

6. Referencias **10**

1. Introducción

1.1 Planteamiento del problema

Implementar una jerarquía de herencia con la máxima profundidad posible para crear cuadriláteros a partir de coordenadas cartesianas. Creando una clase para crear los puntos, y las demás para los cuadriláteros: Trapecio, paralelogramo, rectángulo y cuadrado. Además de realizar una cuestión conceptual a LLMs que permita evaluar críticamente el impacto del modificador protected en herencia.

1.2 Motivación

Comprender la mecánica de la herencia y el encapsulamiento. Entender el diseño de las clases base que repercuten directamente en la extensibilidad del sistema. Analizar el uso riguroso de coordenadas encapsuladas en una clase auxiliar y trasladar fórmulas matemáticas entre clases. Adicionalmente, contrastar las respuestas de la IA sobre las malas y buenas prácticas de diseño en herencia.

1.3 Objetivos

- Implementar una jerarquía de herencia en Java con cinco niveles continuos de profundidad a partir de una superclase base denominada Cuadrilatero.
- Heredar el cálculo del área para las clases derivadas Trapezoide, Paralelogramo, Rectangulo y Cuadrado, aprovechando la sobreescritura de métodos.
- Comparar críticamente las respuestas emitidas por diferentes LLMs sobre las implicaciones del modificador protected.

2. Marco Teórico

La Programación Orientada a Objetos se sustenta en el principio de encapsulamiento. En Java, el control de acceso se articula mediante cuatro niveles de visibilidad: **private**, acceso de paquete (package-private), **protected** y **public** [1]. La especificación oficial del lenguaje (Java Language Specification) establece que un miembro declarado con visibilidad **private** solo es accesible dentro de la clase que lo contiene, mientras que un miembro **protected** puede ser accedido por cualquier clase situada dentro del

mismo paquete y, bajo condiciones específicas, por clases derivadas ubicadas en paquetes externos [2].

Cuando una superclase define la estructura general de un algoritmo y delega pasos concretos a métodos sobreescribibles por las clases derivadas, se puede considerar el uso de métodos protegidos como herramientas de extensibilidad controlada [4]. Finalmente, el principio de composición o agregación permite a una clase incorporar instancias de otras clases como partes de su propio estado evitando jerarquías innecesarias cuando la relación entre componentes corresponde a "HAS-A"[3].

3. Desarrollo

3.1 Actividad 1: Análisis de las respuestas de las LLMs

Análisis de la Respuesta 1

La primera consulta consistió en una pregunta directa que cuestionaba por qué no es una buena práctica emplear **protected** en la herencia. Este planteamiento careció de especificaciones conteniendo un sesgo explícito que inducía al modelo a asumir dicho modificador como un error de diseño categórico.

La respuesta proporcionada por la LLM adoptó un enfoque pedagógico orientado a corregir la premisa inicial. La herramienta aclaró que el uso del modificador no es una mala práctica en sí mismo, sino que su riesgo radica en el abuso sobre variables de estado. Para sustentarlo, presentó una tabla general de modificadores de acceso, expuso un ejemplo en código simulando cuentas bancarias para ilustrar cómo el acceso directo al saldo elude validaciones de negocio y contrastó dicho diseño frente a una alternativa con variables privadas y métodos de retiro.

El modelo explico los conceptos de acoplamiento, pérdida de control del estado y dificultad de mantenimiento, ejemplificando además el uso de **protected** en métodos mediante una clase de reportes. Aunque la explicación resultó conceptualmente acertada, omitió el análisis del acceso a nivel de paquete.

Análisis de la Respuesta 2

La segunda consulta consistió en una instrucción exhaustiva y estructurada de investigación técnica sobre Java, compuesta por nueve apartados. El prompt demandó explícitamente no asumir que **protected** es una mala práctica, requiriendo el contraste de argumentos a favor y en contra, distinciones técnicas entre atributos y métodos,

ejemplos de código comparativos y citas bibliográficas formales y verificables.

La respuesta emitida por la herramienta intentó cumplir con precisión cada uno de los lineamientos solicitados. El modelo fundamentó el marco legal del lenguaje citando la sección 6.6.2 de la *Java Language Specification*, detallando las condiciones que rigen el acceso interpaquete y la visibilidad de paquete. Asimismo, integró el principio de ocultamiento de información de David Parnas y las directrices de diseño de Joshua Bloch para evidenciar que el acceso directo a campos mutables rompe las invariantes de clase y rigidiza el mantenimiento.

En contraparte, justificó el valor de **protected** aplicado a comportamientos a través del patrón *Template Method* de Gamma et al. y desglosó tres escenarios de código que contrastaron campos desprotegidos, encapsulamiento mediante operaciones públicas y puntos de extensión controlados.

Análisis de la Respuesta 3

La tercera consulta estuvo conformada por un único párrafo que encadenó múltiples preguntas consecutivas, interrogando simultáneamente qué hace el modificador, cómo lo realiza a nivel operativo, cuándo es útil y bajo qué condiciones debe o no utilizarse.

La herramienta estableció que la consideración de mala práctica se restringe casi con exclusividad a las variables de estado por comprometer el encapsulamiento y propiciar el problema de la clase base frágil, mientras que en métodos representa un instrumento legítimo para definir contratos internos. Explicó el mecanismo a nivel de compilador mediante la resolución de ámbitos locales y el ligamiento de referencias de memoria, complementando su análisis con una tabla comparativa sobre escenarios de uso recomendado y desaconsejado.

La respuesta no incorporó código ejecutable de apoyo e introdujo una analogía sobre métodos gancho (*hooks*) basada en componentes de la biblioteca React.

Observaciones

Al comparar las tres respuestas, se observó que la profundidad del análisis dependió directamente de la estructura de las instrucciones del prompt. La formulación directa del primer caso propició una explicación elemental y generalista, mientras que la variedad de dudas del tercer caso generó una respuesta conceptual sin ejemplos en código.

Por su parte, el segundo prompt, al imponer restricciones de verificación y

estructura formal, condujo a una investigación técnica extensa. No obstante, las tres LLMs concordaron en una sola conclusión: el uso del modificador *protected* en herencia no debe catalogarse como una mala práctica, sino como un elemento de visibilidad cuyo uso en variables de instancia no cumple adecuadamente el encapsulamiento, pero cuya aplicación en métodos es una técnica válida.

3.2 Actividad 2: Programas de la jerarquía de cuadriláteros

Para resolver el problema planteado se implementó una jerarquía de clases en Java organizada dentro del paquete `mx.unam.fi.die.poo.g7.practicas.practica7`. El diseño estableció cinco niveles continuos, situando en la raíz a la clase `Cuadrilatero`, de la cual se derivó sucesivamente `Trapezoide`, de esta `Paralelogramo`, posteriormente `Rectangulo` y, en el nivel más bajo, `Cuadrado`.

Para la composición de las figuras se creó la clase auxiliar `Punto`, la cual modela una coordenada inmutable en el plano cartesiano mediante dos variables primitivas de tipo `double` (`x` e `y`) declaradas con visibilidad privada, acompañadas de su respectivo constructor y métodos de acceso (*getters*). La clase base `Cuadrilatero` encapsula cuatro referencias privadas a objetos `Punto` denominadas `vertice1`, `vertice2`, `vertice3` y `vertice4`. A través de sus métodos de acceso, `getVertice1()` a `getVertice4()`, la superclase proporciona una lectura controlada para que las clases derivadas inspeccionen las coordenadas sin tener acceso directo a la representación física del estado de la clase padre.

La clase `Trapezoide` extiende a `Cuadrilatero` y define el primer método de cálculo de área. Al recibir los cuatro vértices en su constructor, transfiere las referencias a la superclase mediante la instrucción `super()`. Su método `calcularArea()` extrae las coordenadas necesarias encadenando llamadas a los métodos de acceso de los puntos. La longitud de la base mayor se obtiene de la diferencia horizontal entre los vértices 2 y 1, la base menor a partir de la distancia horizontal entre los vértices 3 y 4, y la altura mediante la diferencia vertical entre los vértices 4 y 1.

Para prevenir inconsistencias causadas por el orden en que se asignan los vértices, la clase implementa sentencias condicionales simples if que evalúan si los resultados intermedios son menores a cero, en cuyo caso invierten algebraicamente su signo para garantizar valores positivos. Tras esta normalización, se realiza la suma de las bases multiplicada por la altura y dividida entre dos, asignando el resultado al atributo privado `area` y retornándolo como un valor flotante.

La clase `Paralelogramo` hereda de `Trapezoide` y sobreescribe el método `calcularArea()`. Dado que un paralelogramo mantiene lados opuestos de igual longitud, el cálculo se

simplifica al producto de la base por la altura. La clase calcula la base horizontal inferior y la elevación vertical aplicando la misma verificación condicional de signo para asegurar magnitudes positivas, devolviendo directamente el producto.

La clase `Rectangulo` desciende de `Paralelogramo`. Puesto que un rectángulo es un caso particular de paralelogramo cuyos ángulos internos son rectos, la fórmula de área, basealtura, permanece invariable. Por ende, la clase sobreescribe el método `calcularArea()` pero no replica la lógica de cálculo; en su lugar, delega la operación invocando explícitamente a su padre mediante la sentencia `super.calcularArea()`.

En el extremo final de la cadena de herencia, la clase `Cuadrado` extiende a `Rectangulo`. Esta clase cuenta con un método auxiliar propio que calcula la dimensión de su base mediante la diferencia horizontal entre los vértices 2 y 1, normaliza su signo ante posibles valores negativos y retorna la elevación al cuadrado de dicha magnitud. La clase sobreescribe formalmente el método `calcularArea()` heredado, delegando su ejecución a su método interno de cálculo.

La verificación del comportamiento del sistema se estructuró en la clase ejecutora `MainFiguras`. El método main define de manera estática los conjuntos de coordenadas cartesianas requeridos para cada figura geométrica. Para el trapezoide se asignaron los puntos (0,0), (8,0), (6,4) y (2,4); para el paralelogramo, las coordenadas (0,0), (6,0), (8,4) y (2,4); para el rectángulo, los vértices (0,0), (6,0), (6,4) y (0,4); y para el cuadrado, las posiciones (0,0), (5,0), (5,5) y (0,5).

Tras instanciar cada objeto derivado mediante la propagación sucesiva de constructores a través de `super()`, el programa invoca las operaciones de cálculo correspondientes e imprime secuencialmente los resultados en la salida. Las operaciones matemáticas procesaron los datos arrojando un área de 24.0 unidades cuadradas para el trapezoide, 24.0 para el paralelogramo, 24.0 para el rectángulo y 25.0 para el cuadrado, reflejando la correspondencia entre la parametrización espacial de los vértices y las fórmulas reducidas implementadas en cada nivel de la jerarquía.

4. Resultados

4.1 Resultados de la Práctica 7

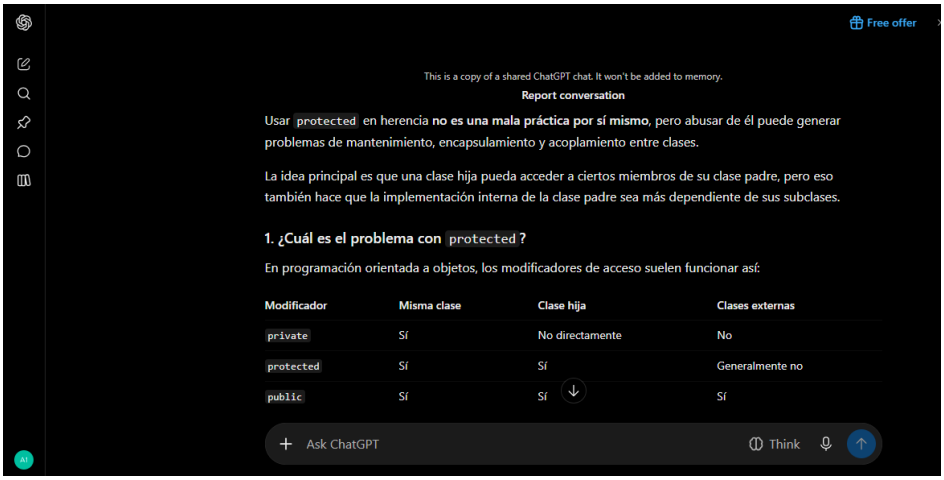


Figura 1: Capturas de pantalla del prompt 1.

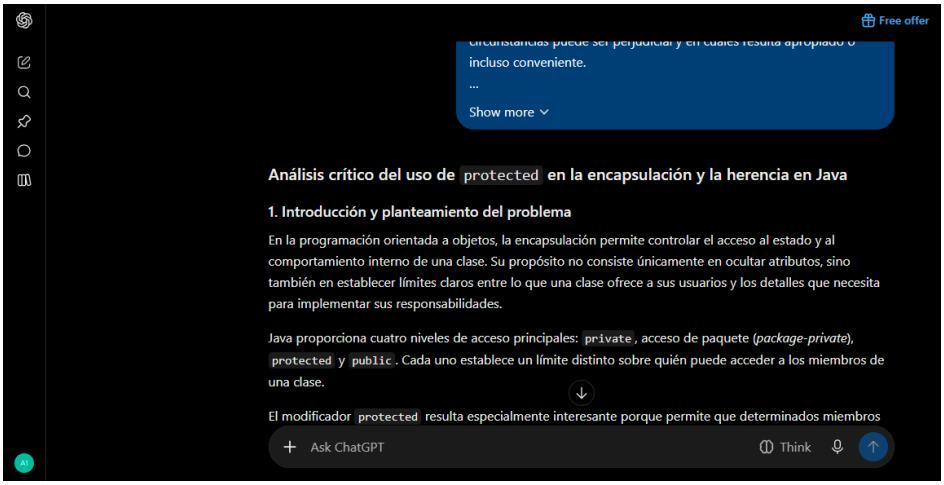


Figura 2: Capturas de pantalla del prompt 2.

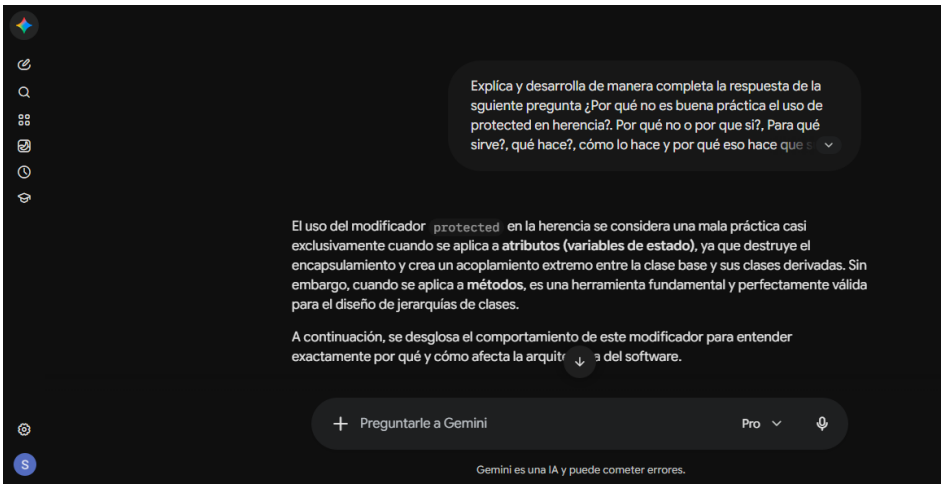


Figura 3: Capturas de pantalla del prompt 3.

Resultados de ejecución

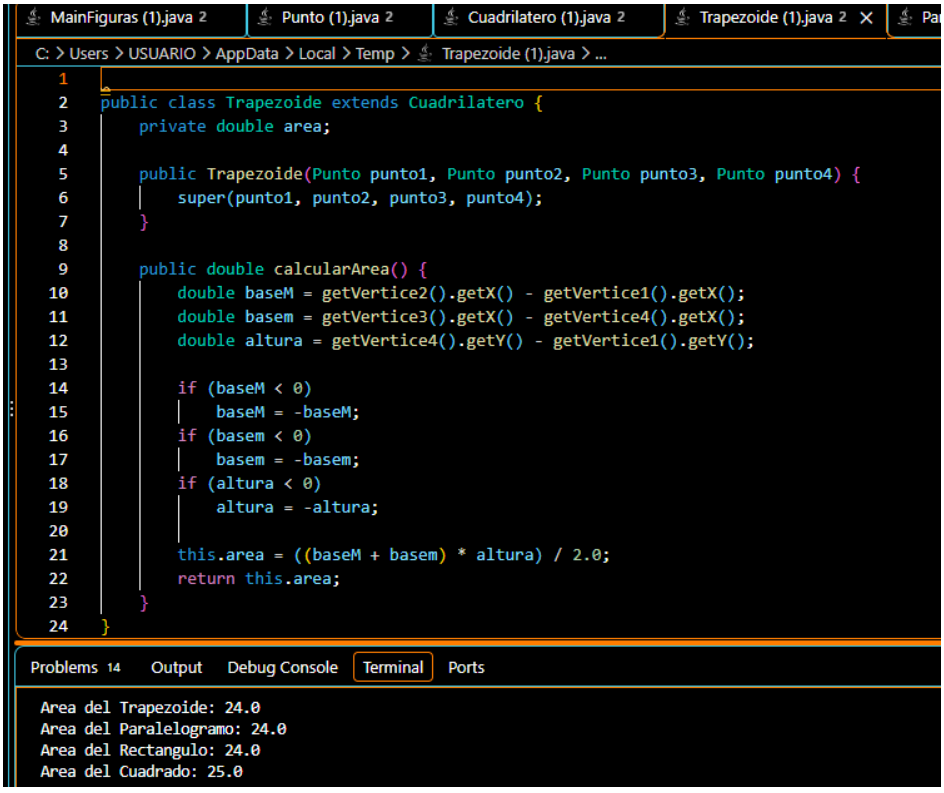


Figura 4: Captura de pantalla de la terminal de ejecución del método principal en MainFiguras

Diagrama 01: Diagrama de Clases UML

Ver imagen 01_Diagrama_Clases_UML.png en GitHub

Diagrama 02: Trapezoide calcularArea

Ver imagen 02_Trapezoide_calcularArea.png en GitHub

Diagrama 03: Paralelogramo calcularArea

Ver imagen 03_Paralelogramo_calcularArea.png en GitHub

Diagrama 04: Rectangulo calcularArea

Ver imagen 04_Rectangulo_calcularArea.png en GitHub

Diagrama 05: Cuadrado calcularArea

Ver imagen 05_Cuadrado_calcularArea.png en GitHub

Diagrama 06: MainFiguras main

Ver imagen 05_MainFiguras_main.png en GitHub

5. Conclusiones

La implementación de la jerarquía de cuadriláteros demostró cómo una cadena profunda puede aprovechar la sobreescritura de métodos y la delegación jerárquica hacia super para eliminar redundancias en el cálculo de áreas. Esto también impidió que las modificaciones de signo o las operaciones aritméticas de las clases derivadas alteraran el estado de los vértices originales, garantizando la inmutabilidad de la figura tras su construcción.

Por otra parte, el contraste de las respuestas teóricas indica que el modificador protected introduce que la exposición de atributos de estado mediante protected traslada la responsabilidad de validar invariantes hacia clases hijas que escapan al control directo de la clase base. En contraposición, cuando protected se aplica a métodos auxiliares dentro de un algoritmo general, se convierte en un recurso valioso para proveer interfaces de extensión controladas.

6. Referencias

[1] Oracle, "Controlling Access to Members of a Class," *The Java™ Tutorials*, 2022. [En línea]. Disponible en:

https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

[2] Oracle, "The Java Language Specification, Java SE 21 Edition," Apartado 6.6, *Access Control*, Oracle, 2022. [En línea]. Disponible en:

https://docs.oracle.com/javase/specs/jls/se21/html/jls-6.html

[3] J. Bloch, *Effective Java*, 3.ª ed., Boston, MA, EE. UU.: Addison-Wesley, 2017. [En línea]. Disponible en:

https://kea.nu/files/textbooks/new/Effective%20Java%20%282017%2C%20Addison-Wesley%29.pdf